



Tailwind

O [Tailwind](#), framework criado em 2017 por Adam Wathan, tem como objetivo otimizar o uso do CSS. É baseado em utilidades, oferecendo várias classes utilitárias, opinativas e de propósito único, e tem como prioridade a facilidade de customização, sendo focado no desenvolvimento *mobile-first*. Estilizar elementos com o Tailwind se assemelha muito com a estilização *inline* (método de escrever CSS dentro do atributo style da tag), porém utilizando classes.

Este framework permite um maior controle na estilização da página, além de possibilitar trabalhar com diversos plugins e ferramentas que facilitam o desenvolvimento, como por exemplo o [PurgeCSS](#), que remove CSS não utilizado no projeto.

Para facilitar a utilização desse framework, você pode instalar no seu Visual Studio Code a extensão [Tailwind CSS IntelliSense](#), criada pela equipe oficial do Tailwind. A extensão oferece uma funcionalidade de linter para avisar e sugerir correções no código, além de auxiliar com os nomes de classes, diretivas e sugerir funções CSS.

```
1 <div class="md:flex">
2   <div class="md:flex-shrink-0">
3     
4   </div>
5   <div class="mt-4 md:mt-0 md:ml-6 bg-re">
6     <div class="uppercase tracking-wide bg-red-100" style="background-color: #f...
7     <a href="#" class="block mt-1 text-l bg-red-200" style="background-color: #f...
8     <p class="mt-2 text-gray-600">Gettin bg-red-300
9   </div>
10 </div>
11
```

```
1 @tailwind utility;
2
3 .example {
4   color: theme('colors.red.50');
5   @apply sm:hover:text-red-800;
6 }
7
8 @screen small {}
9 @variants group {}
10
```



O Tailwind oferece diversas classes utilitárias, sendo algumas delas mostradas abaixo.

- [flex](#): usada para aplicar o flexbox
- [items-center](#): para aplicar a propriedade *align-items: center*;
- [rounded-full](#): para tornar um elemento circular

Uma das maiores críticas ao Tailwind é a de que ele deixa o HTML “sujo”. Para que os elementos não fiquem com muitas classes, dificultando a manutenção e entendimento do código, o Tailwind permite criar classes customizadas que recebem as classes de utilidade.

```
export const button = "text-base font-medium rounded p3"
```

```
<button className={button}>Login</button>
```

Tendo a possibilidade de criar as classes em um arquivo **.js**, é possível também usar interpolações para extrair padrões de texto (conjunto de classes).

```
const flexCol = "flex flex-col items-center justify-between"

export const ul = `${flexCol} md:flex-row gap-2 md:gap-8 font-kalam`
```

Sendo um framework opinativo, mas flexível, possui algumas variantes para a estilização, com valores predefinidos para sombras, deslocamentos, margens, cores, etc. Isso garante a uniformidade em toda a aplicação, resultando em uma melhor experiência do usuário. Mas, caso seja necessário um estilo personalizado, é possível adicionar um tema no arquivo *tailwind.config.js*.

```
module.exports = {
  theme: {
    extend: {
      boxShadow: { '3x1': '0 35px 60px -15px rgba(0, 0, 0, 0.3)' }
    }
  }
}
```

É possível ainda fazer uma estilização personalizada passando um valor arbitrário entre colchetes.

```
<div class="shadow-[0_35px_60px_-15px_rgba(0,0,0,0.3)]">
  <p>Text</p>
</div>
```



Antes do lançamento do compilador [Just-In-Time](#) (JIT), os arquivos Tailwind CSS podiam chegar a ter 15 MB. Embora os estilos não utilizados fossem purgados durante a produção, o mesmo não acontecia durante o desenvolvimento, desta forma as folhas de estilos muito grandes acabavam causando problemas no desenvolvimento das aplicações.

O compilador JIT evita compilar todo o CSS antecipadamente, fazendo isso apenas quando precisamos dele. Como resultado, temos tempos de construção muito rápidos em todos os ambientes e, como os estilos são gerados conforme precisamos deles, não há necessidade de limpar os estilos não utilizados.

Alguns benefícios do compilador JIT são:

- mesma folha de estilo (stylesheet) em desenvolvimento e em produção;
- tempo de construção mais rápido;
- todas as variantes são habilitadas por padrão;
- compilação durante o desenvolvimento é muito mais rápida;
- somente estilos usados são gerados;
- variantes podem ser empilhadas;
- estilização de pseudo-elementos (*::before*, *::placeholder*) e pseudo-classes (*:hover*, *:focus*);
- melhor desempenho das ferramentas de desenvolvimento.